

Belgian Chocolate Problem

Verification Guide

Xuandi Cheng

Proof candidate v1.0.0 — 13 September 2026

1. Claim and conventions

The original problem asks which $\delta > 0$ admit nonzero real polynomials x, y, p , all strictly Hurwitz stable, such that

$$p(s) = (s^2 - 2\delta s + 1)x(s) + (s^2 - 1)y(s), \quad \deg y \leq \deg x.$$

Strictly stable means every complex root has negative real part; nonzero constants are allowed, and no degree or leading-sign normalization is imposed. Put $q = (1 - \delta)/(1 + \delta)$ and $\delta(q) = (1 - q)/(1 + q)$. Write $P(q)$ for original polynomial feasibility. For $0 < q < 1$, $A(q)$ means that a real-symmetric holomorphic function on the unit disc has exactly the simple zero 0 and exactly the simple 1-points $\pm i\sqrt{q}$.

The concrete real $Q = \mathbf{analyticQ}$ is the infimum of the analytic feasible parameters. The main theorem proves $0 < Q < 1/2$ and

$$A(q) \iff Q \leq q, \quad P(q) \iff Q < q \quad (0 < q < 1).$$

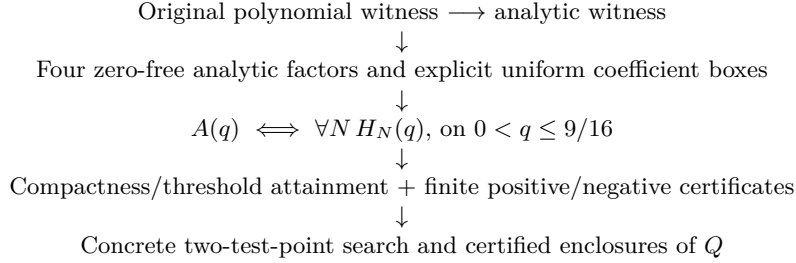
Consequently $\mathcal{A} = (0, \Delta)$, where $\Delta = (1 - Q)/(1 + Q)$, and $\Delta \notin \mathcal{A}$. For every natural precision m , a certified finite search yields rational L, R with

$$L < Q < R, \quad R - L < 2^{-m}.$$

No test deciding whether an arbitrary input equals Q is required.

2. Actual dependency structure

The arrows below indicate mathematical dependencies, not a chronological research narrative.



Two additional branches are essential. Strict analytic slack, Module 3 approximation, and finite Cayley homogenization give a true **OriginalWitness**. Independently, original-polynomial strict improvement plus the forward map excludes the polynomial endpoint. The parameter bound $\delta < 1$ is proved directly from original witnesses. These branches combine with the analytic threshold to give the original feasible-set theorem. The algorithm does *not* itself imply endpoint exclusion.

3. What the release does not claim

No closed form, algebraicity, minimal polynomial, special-function expression, practical complexity guarantee, fixed- N threshold criterion, or generic black-box equality detector is asserted. Independent expert validation is requested; kernel verification does not establish novelty or community acceptance.

4. Ten high-value checkpoints

All names below are actual declarations. Unless a longer prefix is shown, their namespace is `BelgianChocolate`; paths are relative to the repository.

1. Original polynomial to analytic

For $0 < q < 1$, $P(q) \Rightarrow A(q)$, including every actual-degree case.

`polynomialFeasible_implies_analyticFeasible`

`BCPThreshold/PolynomialToAnalytic.lean`. Role: the necessary direction.

2. Uniform all-functions coefficient bounds

For $0 < q \leq (3/4)^2$, every factor coefficient has norm at most the explicit rational C_j , independently of the function and truncation order.

`Schottky.coefficientBound`

`BCPThreshold/CoefficientBounds.lean`. Role: a common compact box.

3. Both directions of all-orders realization

For $0 < q \leq 9/16$, $A(q) \iff \forall N H_N(q)$; the reverse direction constructs compatible infinite coefficients, convergent series, and exact identities.

`analyticFeasible_iff_real_all_levels`

`BCPThreshold/FinalMain.lean`; implementation: `Route1.RealAllOrders.allOrders_realization` and

`Route1.RealAllOrders.analyticFeasible_all_levels`, in `BCPThreshold/RealAllOrdersRealization.lean`. For rational input the matching declaration is `Route1.AllOrders.analyticFeasible_iff_allOrders`.

4. Positive certificate existence

If $0 < r < q < 1$, q is rational, and $A(r)$, a genuine finite rational polynomial certificate exists; exact identity and strict margin are preserved.

`strictAnalyticFeasible_positiveCertificate`

`BCPThreshold/RationalPositiveCertificate.lean`.

5. Negative finite-search completeness

If some rational-parameter finite level is infeasible, some finite stage of the actual Boolean search succeeds. Soundness is separately proved.

`Route1.TrueHierarchy.hierarchyNegative_complete`

`BCPThreshold/RCFRoute1/TrueHierarchy.lean`.

6. Algorithm termination and strict precision

Distinct trisection points ensure finite success without deciding equality to Q ; iterations preserve strict brackets and contract by $2/3$.

`ThresholdData.precisionInterval_correct`

`BCPThreshold/CertifiedAlgorithm.lean`; concrete realization: `analyticThresholdData` in

`BCPThreshold/ConcreteAnalyticCertificates.lean`.

7. Strict-slack original realization

$0 < r < q < 1$ and $A(r)$ imply $P(q)$, via real polynomial Cayley homogenization with exact identity and actual degree.

`strictSlackRealization`

`BCPThreshold/StrictSlackRealization.lean`.

8. Original strict improvement

For $0 < q < 1$, a polynomial witness yields one at some $0 < q' < q$. Affine substitution/coefficient mixing handles leading cancellation.

`polynomialFeasible_strict_improvement`

`BCPThreshold/PolynomialStrictImprovement.lean`.

9. Endpoint separation

$A(Q)$ but $\neg P(Q)$; exclusion depends on checkpoint 8, not on an infimum definition alone.

`endpoint_separation`

`BCPThreshold/EndpointFormalization.lean`.

10. Concrete original-problem main theorem

Combines all parameter ranges, the original feasible interval, the endpoint, the real hierarchy, and arbitrary-precision rational enclosures.

`belgianChocolate_exact_algorithmic_solution`

`BCPThreshold/FinalMain.lean`. No pending interface is a premise.

5. Reproduce the release

Install Git, `elan`, Python 3 and Bash (Git Bash on Windows). Lean 4.33.0 is selected by `lean-toolchain`; Mathlib 4.33.0 is pinned to commit `db584cd6d46c92f209a44c0f1c829460d327499d`.

```
git clone \
https://github.com/chengxuandi/belgian-chocolate-problem-lean-proof-20260910210030-init.git
cd belgian-chocolate-problem-lean-proof-20260910210030-init
git checkout v1.0.0-proof-candidate
lake exe cache get
./verify_bcp.sh
lake build BCPThreshold
```

The cache command obtains compiled dependencies; it does not replace project proof checking. The script builds all BCP modules serially in dependency order, including standalone audits, then the root and `FinalAudit.lean`. It exits on failure. The last explicit build is an optional recheck. Expected final output is `BCP FORMAL VERIFICATION: PASS`.

The checked local source has 57 Lean files including the root, and the audit examines 1,551 imported project declarations. The source scan reports `ZERO_SORRY_CHECK=PASS` and `PROJECT_AXIOM_DECLARATIONS=NONE`. For the main theorem, `#print axioms` reports only

`[propext, Classical.choice, Quot.sound].`

`FinalAudit.lean` checks the required names are real theorem declarations and rejects unexpected transitive axioms, including dependencies hidden through private declarations. Logs are written under `.lake/bcp-verification/`. Nonfatal linter/deprecation warnings may remain.

6. Review routes

Control theory. Begin with `BCPThreshold/Definitions.lean`. Check the original quantifiers, strict stability, parameter inverses, forward Cayley map, strict-slack reverse construction, and actual-degree handling in endpoint improvement. In particular do not replace the original problem by a controller class with extra structural assumptions.

Complex analysis. Inspect the literal `DiscFunction` definition, the four-factor equivalence, uniformity of coefficient bounds, and the compact realization. Pay special attention to compatible coefficients, convergence on the full open disc, inverse identities, and simple prescribed fibres. Lean proves the coefficient estimate by the coarse Schottky/cosine-lift route; the paper gives a classical Hempel-density argument.

Lean/formal methods. Start at `BCPThreshold/FinalMain.lean` and `BCPThreshold/FinalAudit.lean`. Follow the concrete interface bundle, check there is no realization assumption, and reproduce the audit from the tagged source. The original `Admissible` definition is the primary modeling checkpoint, not just the final theorem's name.

7. Important coverage boundary

The main theorem and its actual dependency graph are checked; the release does not claim a line-for-line formalization of every paper assertion. General CAD and the displayed Hempel density theorem are not assumed as project axioms: Lean uses separately proved alternative mechanisms. The paper's algebraic-minimum/non-stabilization discussion is not a separately exported Lean theorem. Finite searches are executable and their success is proved; the total mathematical iterator uses `Nat.find` and remains `noncomputable`. No standalone native threshold calculator is supplied.

AI models assisted exploration, adversarial review, proof engineering and formalization. Independent verification is actively requested. Access the repository, tagged release, and review/contact channel.